



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

VARIABLES, CONVENTIONS & DATA TYPES - SOLUTION

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



1 Variables

A variable in Python is a space in your computer's RAM where you can store results and objects. To create a variable we use an "assignment", that is, an expression with an equality. What is to the left of the equals sign will be the name of the variable where the value is stored, and what is to the right of the equals sign is the value to be stored (*Aplicaciones de Manejo de Datos, 2020*).

For example, let's create the variable `phone_number` and store the value 12345678 in it:

```
phone_number = 12345678 # the symbol = is the assignment operator
phone_number
```

```
## 12345678
```

In python it is not necessary to end a line of code with ; (semicolons), but they do serve a purpose:

```
a = 1; b = 2; c = 3
```

You can also assign values to several variables in a single line of code:

```
a, b, c = 1, 2, 3
```

2 Conventions

1. There are reserved variable names that must not be used because they are reserved by Python:

```
# library
import keyword

# reserved keywords
keyword.kwlist
```

2. Single-line comments are written with #.

```
# this is a single-line comment
```

3. Multi-line comments start with ''' and end with '''.

```
"""
this is
a
multi-line
```

comment
"""

3 Data Types (scalars)

There are four data types or scalars: numeric (**integers** and **floats**), **strings** (text strings) and **booleans** (True & False).

```
'''Basic data types'''
# integers
integer = 2
type(integer) # identification
isinstance(integer, int) # identification

# float
floating = 3.14
type(floating)
isinstance(floating, float)

# string
string = "hello world"
type(string)
isinstance(string, str)

# boolean
boolean = True
type(boolean)
isinstance(boolean, bool)

'''casting'''
# integers
int('1')
int(True)
int(False)

# float
float('1')
float(True)
float(False)

# string
str(1)
str(True)
str(False)

# boolean
bool(1)
bool(0)
```

1. What are the data types in Python? How can I identify them and convert them into other data types?

3.1 Numeric Expressions

1. What numeric operations can I apply?

```
a = 4
b = 3

a + b # addition

## 7
```

```

a - b # subtraction

## 1
a * b # multiplication

## 12
a / b # division

## 1.3333333333333333
a % b # modulo

## 1
a // b # integer part

## 1
a ** b # power

## 64

```

2. What is the result of:

$$1 + 2^3/4 \times 5$$

```

1 + 2 ** 3 / 4 * 5

## 11.0

```

3. Compute the present value of a cash flow of \$1000 over a total of 5 periods. Assume a discount rate of 10%. The present value (PV) is equal to:

$$PV = \sum_{t=0}^T \frac{F_t}{(1+r)^t}$$

```

present_value = 1000 / (1 + 0.1) ** 1 + 1000 / (1 + 0.1) ** 2 + 1000 / (1 + 0.1) ** 3
present_value += 1000 / (1 + 0.1) ** 4 + 1000 / (1 + 0.1) ** 5
print('The present value is: ' + str(present_value))

## The present value is: 3790.7867694084475

```

3.2 Strings

1. Given the following text: “*There are some things money can’t buy, for everything else there’s mastercard*”. Answer the following:
- How many characters does the text contain?

```

text = "There are some things money can't buy, for everything else there's mastercard"
len(text)

## 77

text[0] # first character

## 'T'

text[0:5] # first 5 characters

## 'There'

```

```

text[-1] # last character

## 'd'

```

b. How many times does the letter r appear? How many times does the letter r appear after the “,”?

```

text.count('r')

## 7
text.count('r', text.find(','))

## 5

```

c. What position is “mastercard” at?

```

text.find('mastercard')

## 67

```

d. Replace “mastercard” with “Mastercard”.

```

text.replace('mastercard', 'Mastercard')

## "There are some things money can't buy, for everything else there's Mastercard"

```

2. Write your full name in lowercase, preserving the spaces.

a. Print your full name to the screen in uppercase. (e.g. GUIDO VON ROSSUM)

```

lower_name = "guido von rossum"
lower_name.upper()

## 'GUIDO VON ROSSUM'

```

b. Print your full name to the screen with only the first letters in uppercase. (e.g. Guido Von Rossum)

```

lower_name.title()

## 'Guido Von Rossum'

```

3. Apply the following format¹ to the text: ‘2.45535 dollars is equivalent to \$CLP1808.12’

a. From ‘2.45535’ to 2.45 (two decimal places)

b. ‘dollars’ as text

c. From ‘1808.12’ to 1808 (integer)

```

template = '{0:.2f} {1:s} is equivalent to $CLP{2:d}'
template.format(2.45535, 'dollars', 1808)

## '2.46 dollars is equivalent to $CLP1808'

```

4. What is ‘2’ + ‘25’?

```

'2' + '25' # concatenation, both must be str()

## '225'

```

¹<https://docs.python.org/3/library/stdtypes.html#printf-style-string-formatting>

3.3 Boolean Logic

1. What logical operations can I apply?

Comparison operators:

```
a = 4
```

```
b = 3
```

```
a > b # greater than
```

```
## True
```

```
a < b # less than
```

```
## False
```

```
a == b # equal to
```

```
## False
```

```
a != b # not equal to
```

```
## True
```

```
a >= b # greater than or equal to
```

```
## True
```

```
a <= b # less than or equal to
```

```
## False
```

logical operators:

```
x = True
```

```
y = False
```

```
x and y # True and False = False
```

```
## False
```

```
x or y # True or False = True
```

```
## True
```

```
not x # not True = False
```

```
## False
```

2. Given the following text: “*There are some things money can’t buy, for everything else there’s mastercard*”. Answer the following:

- a. Is the letter ‘z’ present?

```
text = "There are some things money can't buy, for everything else there's mastercard"  
'z' in text
```

```
## False
```

- b. Is the letter ‘m’ present?

```
'm' in text
```

```
## True
```

- c. Is the word ‘visa’ present?

```
'visa' in text
```

```
## False
```

3. Show that De Morgan's law holds.

```
A = True
B = False

not(A and B) == (not(A) or not(B))
```

```
## True
```

```
A = False
B = True

not(A and B) == (not(A) or not(B))
```

```
## True
```

```
A = True
B = True

not(A and B) == (not(A) or not(B))
```

```
## True
```

```
A = False
B = False

not(A and B) == (not(A) or not(B))
```

```
## True
```

4 Appendix

Table 1: De Morgan's Law

A	B	A and B	not(A and B)	not A or not B
True	False	False	not(False) = True	not(True) or not(False) = True
False	True	False	not(False) = True	not(False) or not(True) = True
True	True	True	not(True) = False	not(True) or not(True) = False
False	False	False	not(False) = True	not(False) or not(False) = True